

# SewerStudio

KI-gestuetzte Kanalinspektion

---

Vollstaendiger Programmbericht

Hardware • KI-Modelle • Funktionen • Trainingsplan

Stand: 10. April 2026

Version 3.1 — .NET 8 / WPF / MVVM

Plattform: Windows 11 • Lokal (kein Cloud-Zwang)

# Inhaltsverzeichnis

1. Systemuebersicht
2. Hardware-Architektur
3. KI-Modelle im Detail
4. KI-Pipeline: Ablauf pro Frame
5. Alle Programmfunktionen
6. KI-Trainingsplan (Self-Training)
7. Nutzen und Ergebnisse
8. Umgebungsvariablen

# 1. Systemuebersicht

SewerStudio ist eine Desktop-Anwendung fuer die automatisierte Kanalinspektion. Die Software analysiert Inspektionsvideos mittels einer lokalen KI-Pipeline, erkennt Schaeden nach DIN EN 13508-2 / VSA-KEK 2023 und erstellt vollstaendige Inspektionsprotokolle. Alle Modelle laufen lokal auf der Workstation — keine Cloud-Abhaengigkeit, keine Daten verlassen den Rechner.

Die Anwendung verarbeitet ca. 3000 WinCan/IBAK-IBSK Inspektionsvideos und unterstuetzt den gesamten Workflow: Import, Videoanalyse, Protokollierung, VSA-Bewertung, Sanierungsplanung und Kostenofferte.

## Technologie-Stack

| Komponente | Technologie                               |
|------------|-------------------------------------------|
| Frontend   | WPF / XAML / MVVM (CommunityToolkit.Mvvm) |
| Backend    | .NET 8, C#                                |
| KI-Sidecar | Python FastAPI (YOLO, DINO, SAM)          |
| LLM-Server | Ollama (Qwen2.5-VL, nomic-embed-text)     |
| Datenbank  | SQLite (KnowledgeBase, Projekte)          |
| Video      | ffmpeg/ffprobe (persistent streaming)     |
| Standard   | DIN EN 13508-2 / VSA-KEK 2023 / SIA 405   |

## 2. Hardware-Architektur

### 2.1 Entwicklungs-Workstation

| Komponente | Spezifikation                                   | Rolle                                      |
|------------|-------------------------------------------------|--------------------------------------------|
| CPU        | Intel Core Ultra 9 285K (24 Kerne / 24 Threads) | Post-Processing, ffmpeg, Aggregation       |
| GPU        | ASUS NVIDIA RTX 5090 (32 GB GDDR7)              | YOLO, DINO, SAM, Qwen Inferenz             |
| RAM        | 64 GB DDR5-6400                                 | Projektdaten, Frame-Buffer, Ollama-Context |
| Storage    | NVMe SSD                                        | Video-I/O, SQLite KnowledgeBase            |
| OS         | Windows 11 Pro                                  | WPF-Runtime, CUDA 12+                      |

### 2.2 GPU-Auto-Erkennung (GpuModelSelector)

SewerStudio erkennt beim Start automatisch den verfügbaren GPU-VRAM via `nvidia-smi` und wählt das passende KI-Profil:

| VRAM    | Profil      | Vision-Modell       | NumCtx | Modus                             |
|---------|-------------|---------------------|--------|-----------------------------------|
| ≥ 24 GB | Workstation | Qwen2.5-VL-32B (Q5) | 8192   | Dual-Modell (8B + 32B Eskalation) |
| ≥ 8 GB  | Laptop      | Qwen2.5-VL-7B (Q4)  | 4096   | Single-Modell (nur 8B)            |
| < 8 GB  | Deaktiviert | —                   | —      | Keine KI-Vision                   |

### 2.3 VRAM-Budget pro Modell

| Modell                     | Groesse (Disk) | VRAM (GPU) | Modus              | Verweildauer           |
|----------------------------|----------------|------------|--------------------|------------------------|
| YOLO11m-seg                | ~25 MB         | ~100 MB    | Permanent auf GPU  | Gesamte Session        |
| Grounding DINO 1.5 (SwinT) | ~300 MB        | ~600 MB    | On-Demand          | Nur bei YOLO-relevant  |
| SAM ViT-H                  | ~2.5 GB        | ~2.5 GB    | On-Demand          | Nur bei DINO-Detection |
| Qwen2.5-VL-32B (Q5)        | ~22 GB         | ~26 GB     | Permanent (Ollama) | Keep-Alive 24h         |
| Qwen2.5-VL-7B (Q4)         | ~5 GB          | ~6 GB      | Permanent (Ollama) | Keep-Alive 24h         |
| nomic-embed-text           | ~270 MB        | ~500 MB    | On-Demand (Ollama) | KB-Retrieval           |

**Gesamtbudget Workstation (RTX 5090):** YOLO + DINO + SAM + Qwen32B = ~29 GB. Verbleiben ~3 GB fuer CUDA-Overhead und Ollama-Context. DINO und SAM werden nie gleichzeitig mit Qwen geladen (QualityGate-gesteuert).

### 2.4 Parallelitaets-Einstellungen (Self-Training)

| Setting                  | Default (5090)   | Env-Variable                                 | Beschreibung                                |
|--------------------------|------------------|----------------------------------------------|---------------------------------------------|
| GpuConcurrency           | 6                | SEWERSTUDIO_GPU_CONCURRENCY                  | Parallele Ollama-GPU-Requests               |
| CpuPreExtractParallelism | ProcessorCount-2 | SEWERSTUDIO_SELFTRAIN_PREEXTRACT_PARALLELISM | CPU-Threads fuer PDF-Foto-Vorabextraktion   |
| CaseParallelism          | 3                | SEWERSTUDIO_SELFTRAIN_CASE_PARALLELISM       | Gleichzeitig verarbeitete Inspektionsfaelle |

# 3. KI-Modelle im Detail

## 3.1 YOLO — Pre-Screening und Objekterkennung

YOLO (You Only Look Once) ist das erste Modell in der Pipeline. Es entscheidet in Millisekunden ob ein Frame relevant ist oder uebersprungen werden kann.

| Eigenschaft         | Wert                                                 |
|---------------------|------------------------------------------------------|
| Modell              | YOLO11m-seg (Custom fine-tuned fuer Kanalinspektion) |
| Fallback            | COCO-Gewichte (yolo11m.pt)                           |
| Confidence-Schwelle | 0.25 (konfigurierbar)                                |
| Device              | cuda:0 (Fallback: CPU)                               |
| Inferenzzeit        | 50–200 ms pro Frame                                  |
| Funktion            | Frame-Relevanz, Schadens-Detektion, Bounding-Boxes   |

### CPU-seitiger Qualitaetsfilter (vor GPU-Inferenz):

Bevor YOLO den Frame an die GPU schickt, prueft ein CPU-Filter die Bildqualitaet. Unbrauchbare Frames werden mit 0 ms Inferenzzeit uebersprungen:

| Metrik              | Schwelle | Bedeutung                           |
|---------------------|----------|-------------------------------------|
| Mittlere Helligkeit | < 15     | Zu dunkel (Linsenkappe, kein Licht) |
| Mittlere Helligkeit | > 245    | Ueberbelichtet (Blendung)           |
| Helligkeits-Stdabw. | < 8      | Keine Textur (uniforme Flaeche)     |
| Laplacian-Varianz   | < 5      | Zu unscharf (Bewegung, Defokus)     |

## 3.2 Grounding DINO 1.5 — Text-basierte Detektion

Grounding DINO kombiniert Bildverstaendnis mit Text-Prompts. Es findet Objekte basierend auf natuerlichsprachlichen Beschreibungen — ideal fuer die offene Palette an Kanalschaeden.

| Eigenschaft   | Wert                                                            |
|---------------|-----------------------------------------------------------------|
| Modell        | Grounding DINO 1.5 (SwinT Backbone)                             |
| Architektur   | 6 Encoder + 6 Decoder Layers, 256 Hidden Dim, 8 Attention Heads |
| Box-Schwelle  | 0.30 (konfigurierbar)                                           |
| Text-Schwelle | 0.25 (konfigurierbar)                                           |
| Inferenzzeit  | 200–500 ms pro Frame                                            |
| Einsatz       | Nur bei YOLO-relevanten Frames (On-Demand)                      |

### Text-Prompt (Schaden-Vokabular):

crack . fracture . break . deformation . corrosion . surface damage . erosion . root intrusion . roots . deposit . sediment . buildup . obstacle . blockage . infiltration . water ingress . leak . displaced joint . open joint . offset joint . hole . collapse . missing wall . connection defect . pipe defect . intruding connection . protruding seal

## 3.3 SAM (Segment Anything Model) — Pixel-genaue Segmentierung

SAM verfeinert die DINO-Bounding-Boxes zu pixel-genauen Masken. Daraus werden exakte Masse berechnet: Hoehe, Breite, Flaeche, Querschnittsreduktion.

| Eigenschaft  | Wert                                         |
|--------------|----------------------------------------------|
| Modell       | SAM ViT-H (Vision Transformer Huge)          |
| Gewichte     | ~2.5 GB                                      |
| Input        | Bild + Bounding-Boxes (von DINO)             |
| Output       | Pixel-Masken (RLE-kodiert) + Quantifizierung |
| Multi-Box    | Ja — alle DINO-Detections in einem Request   |
| Inferenzzeit | 100–300 ms pro Frame (alle Boxes)            |

#### Quantifizierte Masse (MaskQuantificationService):

| Mass                         | Einheit    | Verwendung                             |
|------------------------------|------------|----------------------------------------|
| HeightMm / WidthMm           | mm         | Rissbreite, Deformationshoehe          |
| ExtentPercent                | %          | Ausdehnung relativ zum Rohrdurchmesser |
| ClockPosition                | Uhr (1–12) | Lage im Rohrquerschnitt                |
| IntrusionPercent             | %          | Einragung bei Stutzn/Wurzeln           |
| CrossSectionReductionPercent | %          | Querschnittsverengung                  |
| MaskAreaPixels               | px         | Absolute Schadenflaeche                |

### 3.4 Qwen2.5-VL — Vision-Language-Modell

Qwen2.5-VL ist das zentrale Klassifikationsmodell. Es sieht den Frame als Bild, erhält den Kontext der vorherigen Modelle (DINO-Labels, SAM-Masse) und liefert strukturierte VSA-Codes im JSON-Format.

| Eigenschaft    | 32B (Workstation)         | 7B (Laptop)              |
|----------------|---------------------------|--------------------------|
| VRAM           | ~26 GB (Q5 Quantisierung) | ~6 GB (Q4 Quantisierung) |
| Context Window | 8192 Tokens               | 4096 Tokens              |
| Keep-Alive     | 24h (bleibt im VRAM)      | 24h                      |
| Inferenzzeit   | 1–5 Sekunden pro Frame    | 0.5–2 Sekunden           |
| Genauigkeit    | Hoch (Reference-Modell)   | Gut (Fast-Modell)        |
| Einsatz        | Primaer + Eskalation      | Primaer (kein Fallback)  |

#### Funktionen von Qwen2.5-VL:

- VSA-Code-Zuordnung (DIN EN 13508-2 konforme Codes)
- Meter-OSD-Erkennung (liest Meterzaehler aus dem Videobild)
- Material-Erkennung (Beton, Steinzeug, PVC, PE, GFK, Stahl)
- Schweregrad-Bewertung (1–5 nach VSA-Skala)
- Uhrlage-Bestimmung (Position im Rohrquerschnitt)
- Quantifizierung (Ausdehnung in %, Masse in mm)

### 3.5 nomic-embed-text — Embedding-Modell

Fuer die KnowledgeBase wird nomic-embed-text verwendet. Es erzeugt 768-dimensionale Vektoren aus Textbeschreibungen von Schaeden. Damit findet das System aehnliche Faelle aus der Vergangenheit (Few-Shot Retrieval).

| Eigenschaft     | Wert                          |
|-----------------|-------------------------------|
| Modell          | nomic-embed-text (via Ollama) |
| Dimensionen     | 768                           |
| Speicher        | SQLite BLOB (float32-Array)   |
| Similarity      | Cosine Similarity             |
| Top-K Retrieval | Default K=5 (konfigurierbar)  |

### 3.6 QualityGate — Konfidenz-Fusion (Green/Yellow/Red)

Die QualityGate fusioniert alle KI-Signale zu einem Composite-Score. Je nach Ergebnis wird der Fund automatisch akzeptiert (Green), zur Pruefung vorgelegt (Yellow) oder verworfen (Red).

| Signal              | Gewicht | Quelle                   |
|---------------------|---------|--------------------------|
| YOLO Confidence     | 10%     | YOLO Pre-Screening       |
| DINO Confidence     | 15%     | Grounding DINO Detection |
| SAM Mask Stability  | 10%     | SAM Segmentierung        |
| Qwen Vision Conf.   | 15%     | Qwen2.5-VL Analyse       |
| LLM Code Confidence | 20%     | Code-Mapping LLM         |
| KB Similarity       | 10%     | KnowledgeBase Retrieval  |

| Signal             | Gewicht | Quelle                 |
|--------------------|---------|------------------------|
| KB Code Agreement  | 10%     | KB-Code = KI-Code?     |
| Plausibility Score | 10%     | Regelbasierte Pruefung |

| Ampel  | Composite Score | Aktion                        |
|--------|-----------------|-------------------------------|
| Green  | $\geq 0.75$     | Automatisch akzeptiert        |
| Yellow | $0.45 - 0.74$   | Mensch pruefen (Review Queue) |
| Red    | $< 0.45$        | Verworfen / Eskalation        |



## 4. KI-Pipeline: Ablauf pro Frame

Die Multi-Model-Pipeline verarbeitet jeden Frame sequenziell durch bis zu 5 Modelle. Irrelevante Frames werden frueh uebersprungen (Early Exit).

| # | Schritt             | Modell/Tool            | Device | Dauer      | Beschreibung                                     |
|---|---------------------|------------------------|--------|------------|--------------------------------------------------|
| 1 | Frame-Extraktion    | ffmpeg (persistent)    | CPU    | ~0 ms      | PNG-Streaming via stdout, 1 Frame / 3 Sek.       |
| 2 | Base64-Kodierung    | Convert.ToBase64String | CPU    | ~1–5 ms    | Bild fuer HTTP-Transport vorbereiten             |
| 3 | Qualitaetsfilter    | _is_frame_usable()     | CPU    | ~5–10 ms   | Helligkeit, Schaerfe, Uniformitaet pruefen       |
| 4 | YOLO Pre-Screen     | YOLO11m-seg            | GPU    | 50–200 ms  | Relevanz-Entscheidung (skip wenn irrelevant)     |
| 5 | DINO Detection      | Grounding DINO 1.5     | GPU    | 200–500 ms | Text-basierte Schadens-Lokalisierung             |
| 6 | SAM Segmentation    | SAM ViT-H              | GPU    | 100–300 ms | Pixel-genaue Masken + Quantifizierung            |
| 7 | Qwen Klassifikation | Qwen2.5-VL             | GPU    | 1–5 s      | VSA-Code, Meter, Material, Schweregrad           |
| 8 | Temporal Dedup      | ActiveFindingState     | CPU    | ~0.1 ms    | Gleicher Schaden ueber mehrere Frames gruppieren |

### Pipeline-Modi:

| Modus                   | Schritte                                       | Voraussetzung                         |
|-------------------------|------------------------------------------------|---------------------------------------|
| Multi-Model (empfohlen) | Alle 8 Schritte (YOLO+DINO+SAM+Qwen)           | Python-Sidecar auf Port 8100 + Ollama |
| Ollama-Only             | Nur Qwen2.5-VL direkt auf Frame                | Nur Ollama (kein Sidecar noetig)      |
| Auto                    | Versucht Multi-Model, Fallback auf Ollama-Only | Standard-Einstellung                  |

### Post-Processing (nach allen Frames):

Nach der Frame-Analyse durchlaufen die aggregierten Detections eine zweite Phase:

- KB-Retrieval: Aehnliche Faelle aus der KnowledgeBase suchen (Embedding + Cosine Similarity)
- LLM Code-Mapping: Qwen ordnet den endgueltigen VSA-Code zu (mit KB-Beispielen als Kontext)
- QualityGate: Composite-Score berechnen, Ampel zuweisen (Green/Yellow/Red)
- Plausibilitaetspruefung: Regelbasierte Validierung gegen erlaubte VSA-Codes

## 5. Alle Programmfunktionen

### 5.1 Projektverwaltung

SewerStudio organisiert Inspektionsdaten in Projekten. Jedes Projekt enthaelt Haltungen (Kanalabschnitte), Schaechte und zugehoerige Protokolle.

| Funktion                    | Beschreibung                                                   |
|-----------------------------|----------------------------------------------------------------|
| Neues Projekt               | Leeres Projekt mit Metadaten erstellen (Zone, Gemeinde, Firma) |
| Projekt oeffnen             | Bestehendes .awpro-Projektfile laden                           |
| Speichern / Speichern unter | Projekt persistieren (JSON-basiert)                            |
| Wiederherstellungspunkte    | Automatische Snapshots (konfigurierbar)                        |
| Backup / Restore            | Komplettes Projekt-Backup exportieren/importieren              |
| Projektmetadaten            | Zone, Gemeinde, Strasse, Firma, Bearbeiter, Auftraggeber       |

### 5.2 Datenimport (5 Formate)

| Format        | Service                 | Beschreibung                                       |
|---------------|-------------------------|----------------------------------------------------|
| PDF           | PdfImportServiceAdapter | Inspektionsprotokolle (PDF) mit OCR-Unterstuetzung |
| XTF / SIA 405 | XtfImportServiceAdapter | Schweizer Standard-Datenaustausch                  |
| WinCan DB3    | WinCanDbImportService   | WinCan-Projektdateibanken (SQLite)                 |
| IBAK          | IbakExportImportService | IBAK-Inspektions-Exportdateien (Daten.txt)         |
| KINS          | KinsImportService       | KINS-Datenformat (kiDVDaten.txt)                   |

#### Import-Optionen:

- Dry-Run Vorschau: Zeigt was importiert wird, ohne Daten zu aendern
- FillMissingOnly: Nur leere Felder auffuellen, bestehende nicht ueberschreiben
- Import-Bericht: Zusammenfassung mit Found/Created/Updated/Errors

### 5.3 Haltungsverwaltung (Kanalabschnitte)

Die zentrale Datentabelle mit 33 Feldern pro Haltung. Hier werden alle Inspektionsdaten verwaltet, Videos verknuepft und KI-Analysen gestartet.

| Funktion                | Beschreibung                                             |
|-------------------------|----------------------------------------------------------|
| Hinzufuegen / Entfernen | Haltungen manuell erstellen oder loeschen                |
| Video abspielen         | Integrierter Videoplayer mit Zeitcode-Anzeige            |
| KI-Videoanalyse         | Multi-Model-Pipeline auf Video starten                   |
| Protokoll oeffnen       | Inspektionsprotokoll anzeigen und bearbeiten             |
| Video neu verknuepfen   | Fehlende Videolinks manuell oder automatisch korrigieren |
| Massnahmen-Empfehlung   | KI schlaegt Sanierungsmassnahmen vor                     |
| Sanierungs-Optimierung  | KI optimiert Massnahmen ueber alle Haltungen             |
| Kosten oeffnen          | Kostenkalkulator mit Drag-Drop-Interface                 |
| Dossier drucken         | Gesamtbericht als PDF generieren                         |

| Funktion           | Beschreibung                         |
|--------------------|--------------------------------------|
| Live-Suche         | Freitext-Suche ueber alle Felder     |
| Zoom / Zeilenhoehe | Tabellenansicht anpassen (0.5x–2.0x) |

## 5.4 Schachtverwaltung

Verwaltung von Kanalschaechten mit eigenen Feldern und optionalem Protokoll. Gleiche Bedienung wie Haltungen (Hinzufuegen, Entfernen, Suchen, Speichern).

## 5.5 Integrierter Videoplayer

| Funktion             | Beschreibung                                       |
|----------------------|----------------------------------------------------|
| Video-Formate        | MP4, MKV, AVI und weitere (via VLC/LibVLC)         |
| Frame-Navigation     | Frame-fuer-Frame vor/zurueck                       |
| Zeitcode / Meter     | Anzeige von Videozeit und geschaetztem Meter       |
| Zoom und Pan         | Bild vergroessern und verschieben                  |
| Geschwindigkeit      | Wiedergabegeschwindigkeit anpassen                 |
| Vollbild             | Maximierte Ansicht                                 |
| Overlay-Werkzeuge    | Messstab, Defekt-Annotation direkt im Video        |
| Hardware-Dekodierung | GPU-beschleunigtes Video-Decoding (konfigurierbar) |

## 5.6 Protokollverwaltung und VSA-Code-Katalog

| Funktion              | Beschreibung                                                 |
|-----------------------|--------------------------------------------------------------|
| Code-Katalog (VSA)    | Hierarchische Navigation: Gruppe → Code → Char1 → Char2      |
| VSA-Code-Explorer     | Tile-basierte Code-Auswahl mit Live-Validierung              |
| Protokoll-Editor      | Einzelne Beobachtung erfassen/aendern (Meter, Code, Uhrlage) |
| Protokoll-Historie    | Original vs. KI-Revision vergleichen, Audit-Trail            |
| Beobachtungen-Fenster | Alle Beobachtungen einer Haltung filtern und bearbeiten      |
| Streckenschaden       | Start-/Endmeter fuer ausgedehnte Schaeden                    |
| Quantifizierung       | Parameter Q1/Q2 je nach Schadensart (%, mm, Anzahl)          |

## 5.7 VSA-Zustandsbewertung

Berechnet nach VSA-KEK 2023 die Zustandsnoten fuer jede Haltung:

| Note                | Bedeutung                      | Berechnung                                                     |
|---------------------|--------------------------------|----------------------------------------------------------------|
| EZS (Strukturell)   | Baulicher Zustand der Leitung  | Gewichteter Schweregrad aller Schaeden                         |
| EZB (Betrieblich)   | Betriebliche Beeintraechtigung | Ablagerungen, Hindernisse, Querschnitt                         |
| EZD (Dringlichkeit) | Handlungsbedarf                | Kombination aus S und B mit Umfeldfaktoren                     |
| Zustandsklasse      | Gesamtbewertung 0–4            | 0=Sehr gut, 1=Gut, 2=Befriedigend, 3=Schlecht, 4=Sehr schlecht |

## 5.8 Sanierung, Kosten und Offerten

| Funktion                 | Beschreibung                                                            |
|--------------------------|-------------------------------------------------------------------------|
| KI-Massnahmen-Empfehlung | Basierend auf Schadencodes: Inliner, Manschette, Kurzliner, Neubau etc. |
| Sanierungs-Optimierung   | KI optimiert Massnahmen ueber das gesamte Projekt                       |
| Kostenkatalog            | Editierbarer Preiskatalog mit MwSt (JSON)                               |
| Kostenkalkulator         | Drag-Drop: Massnahmen zuweisen, Preise berechnen                        |
| Massnahmen-Templates     | Vordefinierte Sanierungspakete                                          |
| Eigendevis               | Automatische Offerte mit Baugruppen und Positionen                      |
| Export ohne Preis        | Offerte ohne Preisangaben (fuer Ausschreibung)                          |
| Export mit KV            | Vollstaendiger Kostenvoranschlag als PDF/Excel                          |

## 5.9 Druckcenter und Berichtswesen

| Funktion        | Beschreibung                                            |
|-----------------|---------------------------------------------------------|
| Filter          | Nach Eigentuemer, Sanierungsart, Material, Status, Jahr |
| Protokoll-PDF   | Inspektionsprotokoll als PDF exportieren                |
| Dossier         | Gesamtbericht: Daten + Kosten + Massnahmen + Hydraulik  |
| Hydraulik-Druck | Fliessmengen und Druckverluste berechnen und drucken    |
| Statistiken     | Inliner (GFK/Nadelfilz), Manschetten, LEM zaehlen       |
| Kostenausgabe   | Netto, MwSt, Brutto pro Auswahl                         |

## 5.10 Datenexport

| Format            | Beschreibung                                                                |
|-------------------|-----------------------------------------------------------------------------|
| Excel (Haltungen) | Alle Haltungsdaten als Haltungen.xlsx (Template-basiert)                    |
| Excel (Schaechte) | Alle Schachtdaten als Schaechte.xlsx                                        |
| Ordner-Verteilung | Haltungen/Schaechte nach Eigentuerer oder Sanierungsart in Ordner verteilen |
| PDF-Protokoll     | Einzelprotokoll oder Sammelprotokoll als PDF                                |
| Eigendevis Excel  | Offerte als Excel-Arbeitsmappe                                              |

## 5.11 Spezialfunktionen

| Funktion         | Beschreibung                                                       |
|------------------|--------------------------------------------------------------------|
| Kodiermodus      | Live-Erfassung: Overlay-Werkzeuge, Auto-Codevorschlaege, Scan-Modi |
| Bild-Annotation  | Polygon-/Rechteck-Markierung auf Screenshots mit Farbkodierung     |
| Fotomessung      | Laengenmessung in Videos mit Kalibrierung (mm/px)                  |
| Hydraulik-Panel  | Fliessmengen, Druckverluste, Rohrtyp, Neigung                      |
| Medienkonflikte  | Fehlende/mehrdeutige Video-Verknuepfungen automatisch aufloesen    |
| Medien-Suche     | Videos automatisch suchen und Haltungen zuordnen                   |
| Guide-System     | Interaktives Tutorial fuer neue Benutzer                           |
| Fokusmodus (F11) | Sidebar/Menue ausblenden fuer konzentriertes Arbeiten              |
| System-Monitor   | Live-Anzeige: CPU%, RAM%, GPU%, Temperatur                         |
| Dark/Light Mode  | UI-Theme umschaltbar                                               |
| Diagnose-Seite   | Logdateien, Systeminfo, Debug-Optionen                             |

## 6. KI-Trainingsplan (Self-Training)

SewerStudio trainiert sich selbst durch Vergleich mit bestehenden Inspektionsprotokollen. Die KI analysiert Frames blind (ohne Protokoll-Kontext) und vergleicht ihr Ergebnis mit dem Protokoll des Inspektors. Daraus lernt das System kontinuierlich.

### 6.1 Trainingsablauf im Detail

#### Phase 1: PDF-Protokoll-Extraktion (CPU-parallel)

Aus dem PDF-Inspektionsprotokoll werden Beobachtungen und eingebettete Fotos extrahiert. Die Fotos dienen als Ground Truth: Das Bild zeigt den Schaden, der Code im Protokoll ist die korrekte Antwort. Die Extraktion laeuft CPU-parallel (bis 24 Threads).

#### Phase 2: Blinddurchlauf (GPU)

Die KI analysiert jedes extrahierte Foto einzeln — ohne zu wissen was der Inspektor codiert hat. Sie durchlaeuft die volle Pipeline: YOLO → DINO → SAM → Qwen und liefert ihren eigenen VSA-Code-Vorschlag mit Confidence-Score.

#### Phase 3: Deterministischer Vergleich (CPU)

Der DifferenceAnalyzer vergleicht KI-Ergebnis mit Protokoll-Eintrag und klassifiziert:

| Ergebnis     | Bedeutung                                     | Aktion                                 |
|--------------|-----------------------------------------------|----------------------------------------|
| ExactMatch   | Code stimmt exakt ueberein                    | Direkt in KnowledgeBase indexieren     |
| PartialMatch | Code teilweise korrekt (z.B. BAA statt BAB)   | Auto-Approve wenn Score $\geq 0.60$    |
| Mismatch     | Code falsch                                   | In Review-Queue, Mensch entscheidet    |
| NoFindings   | KI hat nichts gefunden, Protokoll hat Eintrag | FN-Analyse, Schwierigkeitsgrad pruefen |

#### Phase 4: Qualitaetskontrolle (QualityGate)

Jedes Sample durchlaeuft die QualityGate. Nur Samples mit genuegend Konfidenz werden automatisch gelernt. Der Rest geht in die Review-Queue:

| Kriterium             | Schwelle    | Beschreibung                      |
|-----------------------|-------------|-----------------------------------|
| Confidence            | $\geq 0.92$ | KI ist sich sehr sicher           |
| QualityGate           | Green       | Composite-Score $\geq 0.75$       |
| KB Agreement          | true        | KnowledgeBase bestaetigt den Code |
| Epistemic Uncertainty | $< 0.15$    | Geringe Modellunsicherheit        |

#### Phase 5: Review-Queue (Human-in-the-Loop)

Samples in der Yellow-Zone werden nach Prioritaet sortiert. Der Mensch prueft und genehmigt oder verwirft. Die Priorisierung nutzt Active Learning:

- 60% Uncertainty Sampling: Samples mit hoechster Unsicherheit zuerst
- 40% Diversity Sampling: Seltene Codes (unterrepraesentierte Kategorien) bevorzugt

#### Phase 6: KnowledgeBase-Indexierung

Genehmigte Samples werden in die KnowledgeBase aufgenommen: Text-Embedding via nomic-embed-text, Speicherung in SQLite. Vor der Indexierung prueft der KbDeduplicationService auf Duplikate (Cosine Similarity  $> 0.95$  = Duplikat).

## Phase 7: Benchmark-Tracking

Nach jedem Trainingsblock wird ein Benchmark-Score berechnet: ExactMatch-Rate, PartialMatch-Rate, Mismatch-Rate. Diese Zeitreihe zeigt ob das System ueber die Zeit besser wird.

## 6.2 Batch-Selbsttraining (ueber Nacht)

Im Batch-Modus verarbeitet SewerStudio automatisch hunderte Inspektionsfaelle. Das System laeuft ueber Nacht und lernt aus dem gesamten Archiv:

- PDF-Fotos aller Faelle vorab extrahieren (CPU-parallel)
- Mehrere Faelle gleichzeitig analysieren (CaseParallelism)
- Automatische KB-Indexierung bei hoher Konfidenz
- Review-Queue fuer unsichere Faelle (morgens pruefen)
- Benchmark-Report am Ende des Batch-Laufs

## 6.3 Few-Shot Learning

Die KnowledgeBase dient als Few-Shot-Speicher. Bei jeder neuen Analyse sucht das System aehnliche Faelle aus der KB und gibt sie als Beispiele in den Qwen-Prompt. So lernt die KI aus vergangenen Entscheidungen:

- Query: Aktueller Schaden (Label, Meter, Schweregrad, Uhrlage, Masse)
- Retrieval: Top-3 aehnliche Faelle per Cosine Similarity
- Kontext: Beispiele werden als In-Context-Learning dem LLM gegeben
- Meter-Gewichtung: Naehere Schaeden werden hoeher gewichtet

## 6.4 Gewichts-Lernen (CategoryWeights)

Die QualityGate-Gewichte werden pro Schadenskategorie optimiert. Wenn z.B. fuer Risse (BAA–BAF) DINO besonders zuverlaessig ist, erhoecht das System automatisch das DINO-Gewicht fuer diese Kategorie. Bayesianische Optimierung mit Validierung.

## 7. Nutzen und Ergebnisse

### 7.1 Zeitersparnis

| Aufgabe                           | Manuell   | Mit SewerStudio KI      | Ersparnis |
|-----------------------------------|-----------|-------------------------|-----------|
| Video-Protokollierung (1 Haltung) | 15–30 Min | 3–5 Min (Review)        | ~80%      |
| VSA-Bewertung (100 Haltungen)     | 1–2 Tage  | Minuten (automatisch)   | ~95%      |
| Sanierungsplanung                 | Stunden   | Sekunden (KI-Vorschlag) | ~90%      |
| Kostenofferte                     | 30–60 Min | 5 Min (Template + KI)   | ~85%      |

### 7.2 Qualitätsverbesserung

- Konsistenz: KI codiert gleiche Schaeden immer gleich (kein Inspekteur-Bias)
- Vollstaendigkeit: Kein Schaden wird uebersprungen (Frame-fuer-Frame Analyse)
- Nachvollziehbarkeit: QualityGate-Score, Evidence-Vektor, KB-Referenzen dokumentiert
- Kontinuierliche Verbesserung: Self-Training-Loop verbessert Genauigkeit mit jedem Batch

### 7.3 Nutzen des Self-Training

| Aspekt          | Ohne Self-Training                | Mit Self-Training                         |
|-----------------|-----------------------------------|-------------------------------------------|
| Genauigkeit     | Basis-Genauigkeit der LLM-Modelle | Steigt mit jedem verarbeiteten Fall       |
| Kontextwissen   | Nur allgemeines Wissen            | Lernt projektspezifische Muster           |
| Seltene Codes   | Oft falsch klassifiziert          | KB-Beispiele verbessern Erkennung         |
| Inspekteur-Stil | Nicht angepasst                   | Adaptiert sich an lokale Konventionen     |
| Aufwand         | Kein Aufwand                      | Laeuft ueber Nacht, Review ~30 Min/Morgen |

### 7.4 Datenschutz und Unabhaengigkeit

- Alle Daten bleiben lokal — keine Cloud, kein Internet noetig
- Alle KI-Modelle laufen auf eigener Hardware
- Kein Vendor-Lock-in: Open-Source Modelle (YOLO, SAM, Qwen via Ollama)
- Volle Kontrolle ueber Modelle, Schwellen und Gewichtungen



## 8. Umgebungsvariablen (Referenz)

SewerStudio wird ueber Umgebungsvariablen konfiguriert. Alle Variablen sind optional — Defaults sind fuer die meisten Setups ausreichend.

### Ollama / KI-Konfiguration:

| Variable                      | Default                | Beschreibung                                |
|-------------------------------|------------------------|---------------------------------------------|
| SEWERSTUDIO_AI_ENABLED        | false                  | KI-Features aktivieren                      |
| SEWERSTUDIO_OLLAMA_URL        | http://localhost:11434 | Ollama-Server-Adresse                       |
| SEWERSTUDIO_AI_VISION_MODEL   | auto                   | Vision-Modell (auto = GPU-basierte Auswahl) |
| SEWERSTUDIO_AI_EMBED_MODEL    | nomic-embed-text       | Embedding-Modell fuer KB                    |
| SEWERSTUDIO_AI_TIMEOUT_MIN    | 5                      | Ollama Request-Timeout (Minuten)            |
| SEWERSTUDIO_OLLAMA_KEEP_ALIVE | 24h                    | Modell im VRAM halten                       |
| SEWERSTUDIO_OLLAMA_NUM_CTX    | 8192                   | Context Window (Tokens)                     |
| OLLAMA_NUM_PARALLEL           | 4                      | Parallele Ollama-Slots                      |

### Pipeline / Sidecar:

| Variable                         | Default               | Beschreibung                      |
|----------------------------------|-----------------------|-----------------------------------|
| SEWERSTUDIO_PIPELINE_MODE        | ollamaonly            | auto / multimodel / ollamaonly    |
| SEWERSTUDIO_MULTIMODEL_ENABLED   | false                 | Multi-Model-Pipeline aktivieren   |
| SEWERSTUDIO_SIDE CAR_URL         | http://localhost:8100 | Python-Sidecar-Adresse            |
| SEWERSTUDIO_SIDE CAR_TIMEOUT_SEC | 300                   | Sidecar Request-Timeout           |
| SEWERSTUDIO_YOLO_CONFIDENCE      | 0.25                  | YOLO Confidence-Schwelle          |
| SEWERSTUDIO_DINO_BOX_THRESHOLD   | 0.30                  | DINO Box-Schwelle                 |
| SEWERSTUDIO_DINO_TEXT_THRESHOLD  | 0.25                  | DINO Text-Schwelle                |
| SEWERSTUDIO_PIPE_DIAMETER_MM     | —                     | Rohrdurchmesser-Override fuer SAM |

### Self-Training:

| Variable                                      | Default     | Beschreibung                    |
|-----------------------------------------------|-------------|---------------------------------|
| SEWERSTUDIO_GPU_CONCURRENCY                   | auto (VRAM) | Parallele GPU-Requests          |
| SEWERSTUDIO_SELFTRAIN_PREE XTRACT_PARALLELISM | CPU-2       | CPU-Threads fuer PDF-Extraktion |
| SEWERSTUDIO_SELFTRAIN_CASE_PARALLELISM        | auto (VRAM) | Gleichzeitige Inspektionsfaelle |

Dieser Bericht wurde automatisch aus der SewerStudio-Codebasis generiert. Stand: April 2026, Version 3.1.